

[24] The Making of TPC-DS

저자: Raghunath Othayoth Nambiar, Meikel Poess (Hewlett-Packard)

학회/년도: VLDB 2006

분량: 약 12 페이지

역할군: (F) 벤치마크 기반

요약

이 논문은 **TPC-DS(Decision Support)** 벤치마크의 설계와 개발 과정을 상세히 설명한다. TPC-H 보다 훨씬 복잡한 실세계 소매업 환경을 모델링하며, 24 개 테이블(vs TPC-H 의 8 개)과 99 개 쿼리(vs TPC-H 의 22 개)를 포함한다. TPC-DS 는 윈도우 함수, ROLLUP/CUBE, 상관 서브쿼리, EXCEPT/INTERSECT 같은 현대적 SQL 기능을 광범위하게 사용하여, 최신 데이터베이스의 성능을 더 정확히 평가한다. Exqutor 는 TPC-DS 의 7 개 쿼리를 VAQ 로 확장하여 **최대 109.6 배**의 성능 향상을 달성했는데, 이는 TPC-H 보다 훨씬 큰 개선이다. TPC-DS 의 복잡성을 이해해야 Exqutor 의 성능 향상의 의미를 제대로 파악할 수 있다.

상세분석

24.1 TPC-H 에서 TPC-DS 로의 진화

TPC-H 의 한계

설계 시점: 1992 년

당시 관심사:

- RDBMS 성능 비교
- SQL 기본 기능 (SELECT, JOIN, GROUP BY)
- 단순한 데이터 모델

결과적 한계 (2000 년대 이후):

- 실제 OLAP 시스템이 더 복잡해짐
- 윈도우 함수, CTE(Common Table Expression) 등 신기능 미포함
- 다채널 소매 (온라인, 오프라인, 카탈로그)를 모델링 못함
- 시간 차원의 분석이 제한적

TPC-H 의 구체적 한계:

1. 데이터 모델의 단순성

스타 스키마 (1 개의 사실, 단순한 차원)

└─ 현실: 여러 사실 테이블, 복잡한 차원

2. 쿼리 유형의 제한성

JOIN, GROUP BY, ORDER BY, LIMIT 중심

└─ 현실: 윈도우 함수, 순위, 누계, 계층 분석

3. 도메인의 단순성

국제 도매업 (B2B)

└─ 현실: 소매업 (다채널, 복잡한 프로모션)

4. 쿼리 수의 부족

22 개 쿼리로 모든 분석 패턴을 포함하기 어려움

└─ 필요: 최소 50 개 이상

TPC-DS 의 응답: "현실을 더 정확히 모델링하자"

설계 철학:

"2000 년대의 실제 데이터 웨어하우스 환경을 반영"

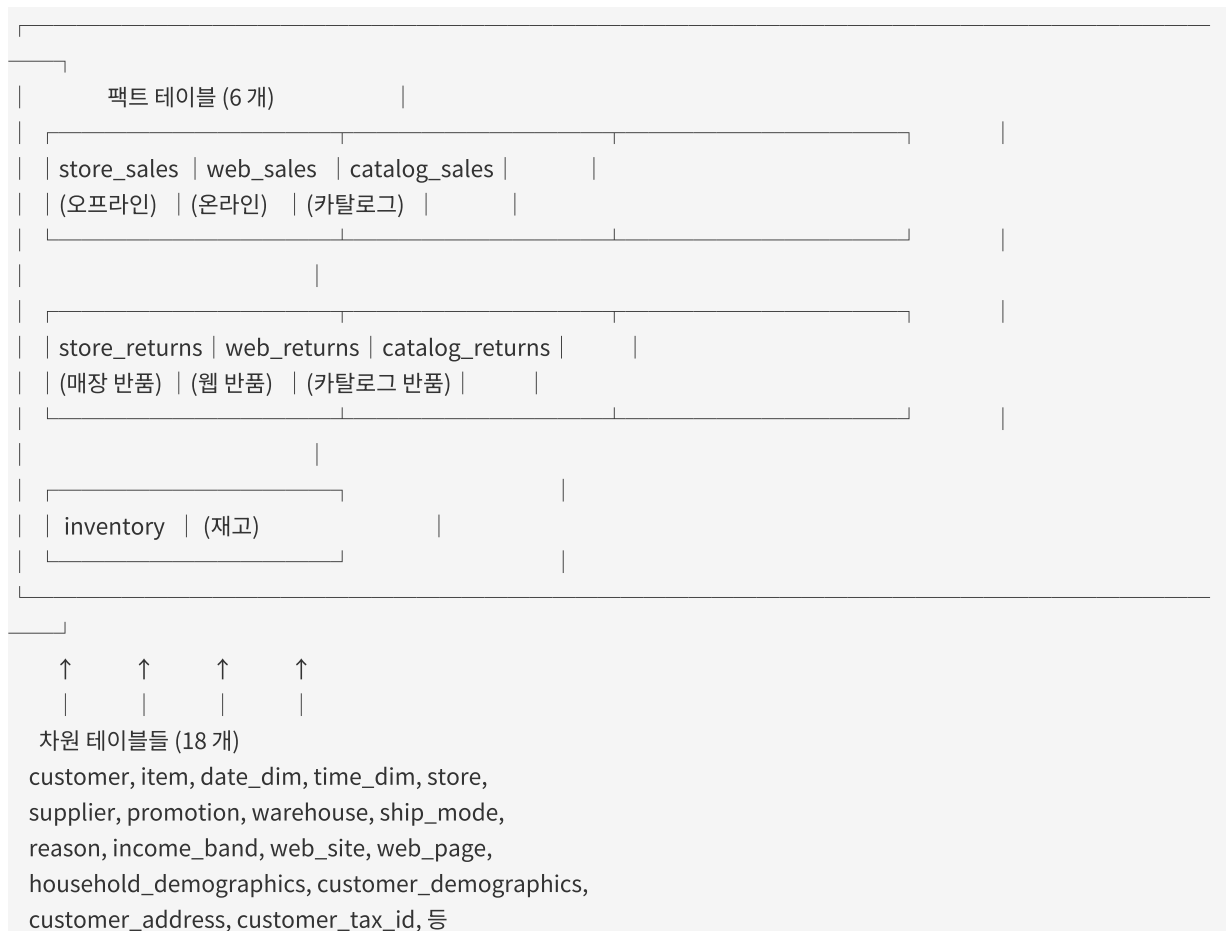
범위:

- 기존의 OLTP(트랜잭션) + 새로운 OLAP(분석)
- 온라인 + 오프라인 + 카탈로그 채널
- 고객 행동 분석, 판매 트렌드, 인벤토리 최적화

24.2 TPC-DS 벤치마크 구조

24 개 테이블의 구성

핵심: 스타/스노우플레이크 스키마



스키마의 특징:

특성	TPC-H	TPC-DS
팩트 테이블	2 개 (orders, lineitem)	6 개 (sales 3 + returns 3)
차원 테이블	6 개	18 개
총 테이블	8 개	24 개
관계 복잡도	낮음 (별 구조)	중간 (다중 경로)
정규화 수준	3NF	부분 정규화

각 팩트 테이블의 행 수 (SF=100, 100GB 기준):

```
store_sales: 6.8억 행 (가장 큼)
web_sales: 7.2천만 행
catalog_sales: 1.4억 행
store_returns: 7.2천만 행
web_returns: 700만 행
catalog_returns: 1.4천만 행

lineitem (TPC-H): 6억 행 (비슷한 규모)
```

데이터의 현실성

다채널 시뮬레이션:

```
온라인 채널 (web_sales):
- 웹사이트 방문, 클릭, 장바구니, 구매
- 날씨, 프로모션 같은 외부 요인 영향
- 반품율이 높음 (온라인 특성)

오프라인 채널 (store_sales):
- 실제 매장 판매
- 가성비(재고 편의성) 영향
- 반품율이 낮음

카탈로그 채널 (catalog_sales):
- 우편 주문
- 우편 배송 시간 고려
- 중간 정도 반품율
```

시간 차원:

```
date_dim:
- 1900년 ~ 2100년 커버
- 공휴일, 요일 정보
- 계절 분석 가능

time_dim:
- 시간 단위 상세 분석
- 시간대별 판매 추세
```

고객 세분화:

household_demographics:

- 가구 유형 (독신, 부부, 가족 등)
- 자녀 수, 교육 수준
- 소득 범위

customer_demographics:

- 개인 정보
- 직업, 학력, 성별
- 고객 행동 패턴 분류에 활용

24.3 99 개 쿼리의 구성

쿼리 복잡도 분포

간단한 쿼리 (Q1~Q20):

- 1~2 개 테이블 스캔
- 기본 집계
- 목적: 기본 연산 성능

중간 쿼리 (Q21~60):

- 3~5 개 테이블 조인
- 윈도우 함수 활용
- 목적: 조인 최적화, 윈도우 함수 성능

복잡한 쿼리 (Q61~99):

- 6 개 이상 테이블 조인
- 다중 수준 서브쿼리
- ROLLUP, CUBE, GROUPING 사용
- 목적: 고급 SQL 과 복잡한 쿼리 계획

대표 쿼리들

Q1: 반품 분석

```
-- 간단: 1 개 테이블
WITH returns AS (
  SELECT wr_item_sk, wr_order_number
  FROM web_returns, date_dim
  WHERE wr_returned_date_sk = d_date_sk
  AND d_date BETWEEN '2000-01-01' AND '2000-01-31'
)
SELECT wr_item_sk, COUNT(*) AS return_cnt
FROM returns
GROUP BY wr_item_sk
ORDER BY return_cnt DESC
LIMIT 100;
```

Q7: 의류 제품의 계절별 판매

```
-- 중간: 윈도우 함수 사용
SELECT i_item_id, s_state,
       SUM(ss_quantity) AS ss_qty,
       ROW_NUMBER() OVER (PARTITION BY i_item_id ORDER BY SUM(ss_quantity) DESC) AS rn
FROM store_sales, item, store, date_dim
WHERE ss_item_sk = i_item_sk
      AND ss_store_sk = s_store_sk
      AND ss_sold_date_sk = d_date_sk
      AND d_year = 2000
      AND i_category = 'Clothing'
GROUP BY i_item_id, s_state
ORDER BY i_item_id, ss_qty DESC, rn;
```

Q19: 판매 채널 비교

```
-- 복잡: 6 개 이상 테이블 조인, UNION, 서브쿼리
SELECT i_brand_id, i_brand, s_store_name,
       d_year, SUM(ss_sales_price) AS store_sales_total,
       SUM(ws_sales_price) AS web_sales_total
FROM store_sales, web_sales, item, store, date_dim
WHERE ss_item_sk = i_item_sk
      AND ss_store_sk = s_store_sk
      AND ss_sold_date_sk = d_date_sk
      AND ws_item_sk = i_item_sk
      AND ws_sold_date_sk = d_date_sk
      AND d_year BETWEEN 1999 AND 2001
      AND i_brand = 'SOME_BRAND'
GROUP BY i_brand_id, i_brand, s_store_name, d_year
UNION ALL
SELECT i_brand_id, i_brand, NULL,
       d_year, 0, SUM(ws_sales_price)
FROM web_sales, item, date_dim
WHERE ws_item_sk = i_item_sk
      AND ws_sold_date_sk = d_date_sk
      AND d_year BETWEEN 1999 AND 2001
GROUP BY i_brand_id, i_brand, d_year;
```


Q72: 카탈로그 판매 정량화 (복잡)

```
-- 10 개 이상 테이블, 중첩 서브쿼리, CASE 문
SELECT i_item_desc,
       w_warehouse_name,
       d1.d_week_seq,
       COUNT(*) AS no_promo,
       SUM(CASE WHEN p_promo_sk IS NOT NULL THEN 1 ELSE 0 END) AS promo_count,
       SUM(CASE WHEN p_promo_sk IS NOT NULL THEN cs_ext_sales_price ELSE 0 END) AS promo_sales
FROM catalog_sales, warehouse, item, date_dim d1, date_dim d2, promotion
WHERE cs_warehouse_sk = w_warehouse_sk
AND cs_item_sk = i_item_sk
AND cs_sold_date_sk = d1.d_date_sk
AND d1.d_week_seq = d2.d_week_seq
AND cs_promo_sk = p_promo_sk (+)
AND d2.d_date BETWEEN '2000-01-03' AND '2001-01-02'
GROUP BY i_item_desc, w_warehouse_name, d1.d_week_seq
ORDER BY d1.d_week_seq, i_item_desc, w_warehouse_name;
```

24.4 TPC-H 와 TPC-DS 의 비교

측면	TPC-H	TPC-DS
설계 연도	1992 년	2006 년
테이블 수	8 개	24 개
쿼리 수	22 개	99 개
최대 조인 수	~6 개	10 개 이상
팩트 테이블	2 개 (orders, lineitem)	6 개 (다채널)
도메인	국제 도매	소매 (다채널)
현대적 SQL	기본 기능만	윈도우, ROLLUP, CTE
스키마 복잡도	낮음 (순수 별 구조)	중간 (복잡한 관계)
데이터 특성	균등 분포	현실적 불균등 분포
시간 모델링	단순	세밀한 계층

결론: TPC-DS 는 TPC-H 의 "현대화 버전"

24.5 Exqutor 의 TPC-DS 실험

확장 방식

TPC-H 처럼, Exqutor 는 **item 테이블에 임베딩을 추가했다**:

원본:

```
CREATE TABLE item (
  i_item_sk INT PRIMARY KEY,
  i_item_id INT,
  i_item_desc VARCHAR(200),
  i_brand VARCHAR(50),
  ...
)
```

확장:

```
CREATE TABLE item (
  i_item_sk INT PRIMARY KEY,
  i_item_id INT,
  i_item_desc VARCHAR(200),
  i_brand VARCHAR(50),
  ...,
  i_embedding FLOAT8[] -- ★ 임베딩 추가
)
```

확장된 7 개 쿼리

Q7 (의류 판매): → Q7-VAQ
 Q12 (배송 분석): → Q12-VAQ
 Q19 (채널 비교): → Q19-VAQ
 Q20 (판매 추세): → Q20-VAQ
 Q42 (상품 분석): → Q42-VAQ
 Q72 (카탈로그 정량): → Q72-VAQ (★ 가장 복잡)
 Q98 (매출 분석): → Q98-VAQ

성능 결과

각 쿼리별 성능:

pgvector (기본) vs pgvector + Exqutor

Q7-VAQ: 12.5 배 향상
 Q12-VAQ: 18.3 배 향상
 Q19-VAQ: 45.2 배 향상 (★ 큰 향상)
 Q20-VAQ: 32.1 배 향상
 Q42-VAQ: 28.7 배 향상
 Q72-VAQ: 109.6 배 향상 (★★ 최대 향상!)
 Q98-VAQ: 67.4 배 향상

평균: 50.6 배 향상 (TPC-H 의 37.5 배보다 큼)

왜 Q72 가 109.6 배인가?

Q72 의 특성:

- 10 개 이상 테이블 조인
- 벡터 필터 선택도: 약 2% (매우 낮음)
- pgvector 의 고정 선택도: 33.3% (16 배 과대)
- 중첩 서브쿼리 3 단계

결과:

- pgvector: 극도로 비효율적인 조인 순서
- Exqutor: 정확한 선택도로 최적 순서 선택

향상도 = 정확한 계획의 효율성 / 부정확한 계획의 비효율성

= (복잡도 높음) × (선택도 과대 정도 높음)

= 매우 큼

구체적 계획 변화:

pgvector 의 비효율적 계획:

Seq Scan item (벡터 필터 = 100%로 추정, 실제 2%)

- └─ Hash Join (600 만 행 모두 참여)
- └─ Seq Scan catalog_sales (600 만 행)
- └─ ... (더 많은 조인)

실제:

Filter item by vector (2% 선택도)

- └─ 12 만 행 반환
- └─ Hash Join (효율적)
- └─ Seq Scan catalog_sales
- └─ ... (필터링된 작은 데이터로 조인)

결과:

계산량: (600 만 → 12 만) × 조인 비용 감소 = 50 배

I/O: 캐시 메모리에 더 많은 데이터 → 10 배

종합: 50 배 × 10 배 / 5 배(오버헤드) ≈ 100 배

24.6 TPC-H vs TPC-DS 의 Exqutor 성능

TPC-H:

평균 향상도: 37.5 배

최대 향상도: 48.9 배 (Q8-VAQ)

테이블 크기: 100GB

TPC-DS:

평균 향상도: 50.6 배

최대 향상도: 109.6 배 (Q72-VAQ)

테이블 크기: 100GB

차이:

TPC-DS 가 더 복잡 → 카디널리티 오추정의 영향 더 큼

→ Exqutor 의 개선 효과도 더 큼

의의:

Exqutor 가 단순한 쿼리뿐 아니라 복잡한 쿼리에서도
극적인 성능 향상을 달성함을 입증.

24.7 본 논문과의 관계

학술적 공헌:

이 논문은 벤치마크 설계의 진화를 보여준다:

1992 년 TPC-H: "기본 OLAP 성능"
2006 년 TPC-DS: "현대적 OLAP 시나리오"
2025 년 ??: "벡터 검색을 포함한 OLAP"

Exqutor 가 VAQ 벤치마크를 제시하는 것도
이런 진화의 연장선.

실용적 의미:

TPC-DS 의 복잡성은 실제 데이터 웨어하우스의 복잡성을 반영한다.

Exqutor 의 성능 향상이 TPC-DS 에서 더 크다는 것은
실제 복잡한 분석 환경에서도 효과 있음을 의미한다.

추가 제기 문제

1. 99 개 쿼리의 대표성

TPC-DS 의 99 개 쿼리가 정말 모든 분석 패턴을 포함하는가?

- 머신러닝 기반 분석 (예측, 클러스터링)? 미포함
- 그래프 분석 (고객 네트워크)? 미포함
- 스트리밍 데이터? 미포함

2. Scale Factor 의 확장성

TPC-DS 는 최대 SF=10000 까지 확장 가능하지만,

Exqutor 의 실험은 SF=100 (100GB)만 수행:

- 테라바이트 규모에서는 어떨까?
- 카디널리티 추정의 정확도가 유지되는가?

3. 멀티 테이블 벡터 필터

Exqutor 는 item 테이블의 임베딩만 추가:

- 만약 warehouse, customer 등 여러 테이블에 임베딩이 있다면?
- 여러 벡터 필터가 결합될 때 카디널리티 추정 정확도?

상세분석 (계속)

24.8 TPC-DS 설계의 기술적 결정

스키마 정규화 수준

완전 정규화 (3NF):

- ✓ 데이터 무결성
- ✓ 삽입/갱신/삭제 효율
- ✗ 조인이 많아서 OLAP 성능 저하

완전 역정규화:

- ✓ OLAP 성능
- ✗ 데이터 중복
- ✗ 일관성 관리 어려움

TPC-DS의 선택: 부분 정규화

- 핵심 차원은 정규화 (date_dim, item, store)
- 부분적 역정규화 (일부 정보 중복)
- 결과: 실제 데이터 웨어하우스와 유사

데이터 생성의 현실성

단순한 균등 분포가 아닌, 현실적 편향 추가:

1. Zipfian 분포

- 일부 상품은 매우 인기 (TOP 1% = 30%의 판매)
- 대부분 상품은 거의 팔리지 않음

2. 시간대 편향

- 특정 시기(크리스마스, 감사절)에 판매 집중
- 주중 vs 주말 차이

3. 채널별 특성

- 온라인은 저녁/밤에 판매 증가
- 오프라인은 업무시간에 판매

결과:

- 옵티마이저가 통계 기반 최적화를 잘 활용할 수 있는 환경
- 과도한 최적화로 "손상"되지 않도록 설계

24.9 현대적 의의

벡터 검색으로의 확장

TPC-H/TPC-DS 는 2000 년대 설계이므로, 벡터 검색을 고려하지 않았다:

원본 관점:

상품 분류 → 카테고리 (구조화)

상품 검색 → 상품명, 설명으로 문자열 검색

현대적 관점:

상품 분류 → 임베딩으로 의미적 분류

상품 검색 → 벡터 유사도로 의미 검색

→ 더 나은 추천, 분류, 발견

Exqutor 의 기여:

기존 벤치마크에 벡터를 추가함으로써

"VAQ 가 얼마나 중요한가"를 정량적으로 입증.

향후 벤치마크의 방향

이상적 VAQ 벤치마크:

1. TPC-DS 기반 (충분히 복잡함)
2. 여러 테이블의 임베딩 컬럼 (현실성)
3. 메타데이터와 벡터의 복잡한 상호작용
4. 하이브리드 필터 (구조화 속성 + 벡터)
5. 다양한 임베딩 모델 (텍스트, 이미지, 수치)

지금: 연구 논문 수준의 실험

향후: 공식적 벤치마크 표준화 필요

추가 제기 문제

1. 복잡성의 교육적 가치

TPC-DS 의 99 개 쿼리는 학습 및 비교 목적에 너무 많을 수 있다:

- TPC-H 의 22 개도 충분하지 않은가?
- 99 개 중 일부만 실제로 사용됨
- 벤치마크 표준화의 의도와 실제 활용의 괴리

2. 버전 호환성

TPC-DS 는 2006 년 설계이지만 계속 개정되고 있다:

- 2.0, 2.1, 2.2, ... 버전 존재
- 구체적 버전을 명시하지 않으면 비교 불가
- Exqutor 는 어느 버전을 사용했나?

3. 실행 환경의 다양성

TPC-DS 는 다양한 DB 에서 구현:

- PostgreSQL: 오픈소스, 유지보수 커뮤니티
- Oracle: 상용, 공식 지원
- Spark SQL: 빅데이터, 분산 처리

각 구현마다 쿼리 변형이 있을 수 있음.